

PROJECT PHOENIX

Vulnerability Assessment → Final Report

Cyber Security Sub-Team → CY015

Hardhat Enterprises | Trimester Engagement

Compiled by

Himanshu Khanna

Sub-Team Leader, CY015

Contributing Team Members

Himanshu · Rasanjana · Hayden · Inderjeet

Report Date: 24 May 2026

Table of Contents

1. Executive Summary
2. About This Report
3. The Team and Who Did What
4. Scope and Test Environment
5. Overall Results → Summary Table
6. Detailed Findings → Authentication Tests (AT-01 to AT-05)
7. Detailed Findings → Threat, Risk and Dashboard Tests (RT-01 to RT-10)
8. Detailed Findings → Hazard Endpoint Tests (HZ-01 to HZ-03)
9. Detailed Findings → Metadata Endpoints and Infrastructure (Inderjeet's Work)
10. Vulnerabilities Found → Priority List
11. Recommendations
12. Backend Team Coordination
13. Conclusion
14. Acknowledgements

1. Executive Summary

This is the final vulnerability assessment report for the work the CY015 sub-team carried out on the Phoenix backend during this trimester. The goal was simple go through the Phoenix API the way a real attacker would, find what is broken, and write up what we found with evidence.

Across the team we ran 28 test cases in total, covering five different areas of the application. I personally ran the authentication tests (AT-01 to AT-05). Rasanjana took the threat intelligence, risk assessment and dashboard endpoints (RT-01 to RT-10). Hayden handled the data ingestion and hazard endpoints, the hazard ones being the three you will see as HZ-01 to HZ-03 in this report. Inderjeet did the infrastructure stabilisation work and the metadata endpoint testing across six production routes.

The headline findings are not pretty. The login endpoint has no rate limiting at all I ran 15 wrong-password attempts in a row from the same account and the system never once slowed me down or locked the account. The data ingestion endpoint accepts unauthenticated requests (Hayden flagged this in his earlier ingestion testing). And the user registration endpoint accepts the role field straight from the client which means an admin can spin up as many other admin accounts as they want just by putting role admin in the body.

There is also some good news. The backend team fixed a couple of things while we were testing. The threats endpoint was returning data without any token earlier in the testing window that has been patched and it now correctly returns 401. The logout endpoint properly invalidates refresh tokens once you call it with the right URL format. SQL injection attempts against the login endpoint and all six metadata endpoints get blocked before they reach the database, which is exactly what should happen.

Out of 28 tests, two failed outright, five came out as partial passes, fifteen passed, and six were marked as deferred because the relevant backend services were not deployed during our testing window. The deferred ones are not a finding by themselves but they are a real coverage gap that the next team picking this up will need to close.

All findings, evidence and recommendations are laid out in the sections below.

2. About This Report

This document brings together every test case run by the CY015 sub-team and presents the results in one place. It is meant to be read by the project leaders, the backend development team, and any future CY015 team that continues this work.

Each member of the team produced their own detailed test document during the trimester and uploaded it to GitHub. Those individual documents have the full screenshots and Postman captures. This report does not repeat all of that instead it summarises each finding, links it back to the original test case, and pulls everything into one priority view so it is clear what needs fixing first.

Where there is disagreement between what we tested and what the backend team's investigation found later, I have updated the result. This happened twice AT-02 (logout) and AT-04 (threats endpoint) both moved from FAIL to PASS during the testing window because of the back-and-forth with Trung from the backend team. Both are explained in the relevant sections.

3. The Team and Who Did What

Four people contributed to this work. The split was based on what each person had time for and which part of the Phoenix backend made sense to give them.

Name	Role	What They Covered
Himanshu Khanna	Sub-Team Leader	Authentication attack testing (AT-01 to AT-05), backend environment setup, document review, final report compilation
Rasanjana	Tester	Threat intelligence, risk assessment and dashboard endpoint testing (RT-01 to RT-10)
Hayden	Tester	Data ingestion endpoint testing (DI tests) and hazard endpoint testing (HZ-01 to HZ-03)
Inderjeet	Tester	Infrastructure stabilisation work, six production metadata endpoint security tests (SQL injection and unauthenticated access)

As Sub-Team Leader my job was to make sure everyone had what they needed to test, review their documents once they sent them in, push back where the evidence was thin, and stay in regular contact with the backend team and our mentors to keep the project moving.

4. Scope and Test Environment

4.1 What Was In Scope

All testing was carried out against the Phoenix backend API running locally via Docker Compose. We were testing the merged dev branch as it stood at the time of testing. Frontend code was out of scope.

The endpoint groups covered across the team:

- Authentication endpoints → login, logout, refresh, register
- Threat data endpoints → list and detail
- Risk assessment endpoints → list and detail
- Dashboard endpoints → overview and charts
- Data ingestion endpoints → stream
- Hazard endpoints → list and detail
- Metadata endpoints → six production routes under `/api/users/meta`

4.2 Methodology

The testing followed the STRIDE threat modelling framework. Every test ID in this report ties back to a STRIDE threat that was already identified in the CY005 STRIDE analysis document. So when you see ST-03 or ST-17 referenced in a test case, that is pointing back to the threat we already documented earlier in the project. Nothing in this testing was random we tested the things we said we were going to test.

For each test the tester would identify the endpoint, define the attack tactic, send the request through whatever tool made sense (Postman, a script, Burp Suite), capture the response with a screenshot, and judge it against what should have happened. Status codes, response body content, and the time it took to get a response were all factored in.

4.3 Tools Used

- Postman → most of the manual testing went through here
- Python with the requests library → custom brute force script for the login endpoint (15-attempt loop)
- PowerShell Invoke-RestMethod → used for the metadata SQL injection sweep across the six endpoints

- Burp Suite → interception and parameter modification for a small number of tests
- Docker Desktop with WSL 2 → running the Phoenix backend locally
- Supabase → the database. The backend team provided test accounts (analyst_test1 and admin1)

4.4 Local Test Environment Details

Local URL	http://localhost:3001
Backend setup	Docker Compose with 6 containers api-gateway, user-service, ingestion-service, storage-service, rabbitmq, redis
Database	PostgreSQL via Supabase connection string
Test accounts	admin1 (admin role) and analyst_test1 (analyst role) both provided by Trung from the backend team
Test dates	May 2026

5. Overall Results - Summary Table

This is every test case we ran, sorted by tester and test ID. The status column is colour coded — red for failures, yellow for partial passes, green for passes, grey for deferred.

Test ID	Tester	Endpoint	Attack Tactic	Status
AT-01	Himanshu	/api/users/auth/login	Brute force no rate limit	FAIL
AT-02	Himanshu	/api/users/auth/logout/{userId}	Token replay after logout	PASS
AT-03	Himanshu	/api/users/auth/register	Privilege escalation via role field	PARTIAL PASS
AT-04	Himanshu	/api/users/threats	Unauthenticated access	PASS
AT-05	Himanshu	/api/users/auth/login	SQL injection on login	PASS
RT-01	Rasanjana	/api/users/threats	SQL injection via query param	DEFERRED
RT-02	Rasanjana	/api/users/threats/:threatId	RBAC End User access	DEFERRED
RT-03	Rasanjana	/api/users/threats	Unauthenticated access	PASS
RT-04	Rasanjana	/api/users/risk-assessments	Parameter tampering	DEFERRED
RT-05	Rasanjana	/api/users/risk-assessments/:id	RBAC End User access	DEFERRED
RT-06	Rasanjana	/api/users/risk-assessments/:id	IDOR	DEFERRED
RT-07	Rasanjana	/api/users/dashboard/overview	Unauthenticated access	PASS
RT-08	Rasanjana	/api/users/dashboard/overview	Expired token	PASS
RT-09	Rasanjana	/api/users/dashboard/charts	RBAC End User access	PARTIAL PASS
RT-10	Rasanjana	/api/users/dashboard/overview	Authenticated analyst access	PARTIAL PASS
HZ-01	Himanshu	/api/users/hazards	Unauthenticated access	PARTIAL PASS
HZ-02	Hayden	/api/users/hazards?id=' OR '1'='1	SQL injection via query param	PARTIAL PASS
HZ-03	Hayden	/api/users/hazards/{id}	IDOR	PARTIAL PASS
MD-01	Inderjeet	/api/users/meta/locations	SQL injection sweep	PASS
MD-02	Inderjeet	/api/users/meta/event-statuses	SQL injection sweep	PASS
MD-03	Inderjeet	/api/users/meta/linked-event-types	SQL injection sweep	PASS
MD-04	Inderjeet	/api/users/meta/seasons	SQL injection sweep	PASS
MD-05	Inderjeet	/api/users/meta/reference-days	SQL injection sweep	PASS
MD-06	Inderjeet	/api/users/meta/reference-times	SQL injection sweep	PASS
MD-07	Inderjeet	All six /meta endpoints	Unauthenticated access loop	PARTIAL PASS

Of 25 active tests, 15 passed cleanly, 6 came out as partial passes (mostly because the endpoint returned 500 errors instead of the proper status code), and 2 failed outright AT-01 (no rate limiting on login) and the ingestion endpoint accepting unauthenticated requests which Hayden documented in his data ingestion testing earlier in the trimester. Six tests on threat intelligence and risk assessment endpoints were deferred because those backend services were not deployed in the test environment.

6. Detailed Findings - Authentication Tests

This section covers the five tests I ran personally against the Phoenix authentication endpoints. I picked these five because they map directly to the most common attack categories against any login system brute force, token replay, privilege escalation, broken access control, and SQL injection.

6.1 AT-01 → Brute Force Attack on Login

So the idea here was simple — keep sending wrong passwords to the login endpoint and see whether the system would eventually stop me, lock the account, or at least slow me down. If a login endpoint has no rate limiting in place then anyone with a username and a wordlist can sit there guessing passwords forever.

Test ID	AT-01
STRIDE Reference	ST-03 - Spoofing
Endpoint	POST /api/users/auth/login
Tool Used	Python script (custom 15 attempt loop)
Account Targeted	analyst_test1 (real test account)
Expected Result	Should have blocked me after a few wrong attempts either with a 429 Too Many Requests or by locking the account
Actual Result	Got 401 Invalid username or password every single time for all 15 attempts. The system never once slowed me down, never threw a 429, never locked the account. Each attempt got the same response in roughly the same time as the first one.
Status	FAIL
Fix Needed	Needs rate limiting on the login endpoint. After about 5 wrong attempts it should start blocking either by returning 429 or by introducing a delay. A CAPTCHA after a few failures would also help.

Why this matters: Combined with a leaked credentials list or even just predictable usernames (admin1, admin2, etc.) this gives an attacker a clean path into the system. Brute force is one of the simplest attacks there is and the only reason it usually doesn't work is because most systems block it. Phoenix currently does not.

6.2 AT-02 → Token Replay After Logout

The idea here was to check what happens if someone grabs a login token and tries to keep using it even after the original user has logged out. If the system does not properly kill tokens on logout then a stolen token stays valid forever which is a real problem.

This one was interesting because the first time I tested it I got it wrong. I was calling `/api/users/auth/logout` without the `userId` and getting a 404. So I marked it FAIL because as far as I could tell the logout endpoint didn't exist. Trung from the backend team came back to me with screenshots showing that the correct URL is actually `/api/users/auth/logout/{userId}` — with the `userId` as a path parameter. Once I tested with that format the endpoint worked exactly like it should have.

Test ID	AT-02
STRIDE Reference	ST-03 - Spoofing
Endpoint	POST <code>/api/users/auth/logout/{userId}</code> and POST <code>/api/users/auth/refresh</code>
Tool Used	Postman
Expected Result	After logout the refresh token should be invalidated. Using it again should return 401
Actual Result	Logout endpoint returned 200 OK Logged out successfully. When the same refresh token was used again after logout it returned 401 Invalid refresh token, confirming tokens are properly invalidated on logout
Status	PASS
Fix Needed	Nothing logout and token invalidation are working correctly. One small thing worth mentioning: the path-parameter <code>userId</code> format is a bit unusual and may confuse new testers. The earlier 404 result from my own testing is proof of that.

6.3 AT-03 → Privilege Escalation via Role Field

The idea here was to see if I could just put role admin in the signup request body and get admin access without anyone server-side checking. Some systems blindly accept whatever the client sends which would be a huge problem because it means anyone with access to the registration endpoint can make themselves an admin.

Test ID	AT-03
STRIDE Reference	ST-25 - Session Exploitation
Endpoint	POST <code>/api/users/auth/register</code>
Tool Used	Postman

Expected Result	Should either ignore the role field or reject it. Account should come back as a regular user not an admin.
Actual Result	Two scenarios tested. With an analyst token in the Authorization header got 401 Invalid token, access correctly denied (analysts cannot create new accounts). With an admin token in the Authorization header got 201 Created and the system accepted role admin straight from the request body and created a brand new admin account. The role value was never validated or ignored server side.
Status	PARTIAL PASS
Fix Needed	The backend should never trust the role field sent by the client. Role assignment should always be handled server side only, regardless of what the client sends. If the system needs a way to promote users to admin, that should be a separate admin-only endpoint with proper audit logging.

Why this matters: Right now any admin can create unlimited new admin accounts by just adding one line to the request body. This breaks the audit trail and makes it easy for a compromised admin account to spawn shadow admins that are hard to detect later. The endpoint validates that the role string is in an allowed list, but the choice itself sits with the client which is the underlying problem.

6.4 AT-04 → Unauthenticated Access to Threat Data

This was checking whether someone who is not logged in at all can still pull sensitive data from the system. Any endpoint that returns private data should refuse to work without a valid login token.

Like AT-02 this one had a journey. When I first ran it the threats endpoint was returning 200 OK with data when called with no token at all total information disclosure. After the backend team pushed a fix and I retested, the endpoint now correctly returns 401.

Test ID	AT-04
STRIDE Reference	ST-17 - Information Disclosure
Endpoint	GET /api/users/threats
Tool Used	Postman
Expected Result	Should have refused with 401 since there was no login at all
Actual Result	The threats endpoint returned 401 Unauthorized with the message No token provided. Unauthenticated access is properly blocked.
Status	PASS

Fix Needed	Nothing — authentication middleware is working correctly here now. Worth doing a wider audit of other <code>/api/users/*</code> routes to make sure the middleware is applied consistently from one central place, since this endpoint was missed initially.
-------------------	--

6.5 AT-05 — SQL Injection on Login

So basically what I was doing here was typing database commands into the login fields to see if the system would get confused and let me in. This is one of the oldest tricks in the book and a lot of systems still fall for it if input is not properly handled.

Test ID	AT-05
STRIDE Reference	ST-10 - Database Tampering
Endpoint	POST <code>/api/users/auth/login</code>
Tool Used	Postman
Payload	' OR '1'='1 in the username field, random value in password field
Expected Result	Should reject it straight away an injection like this should never result in a successful login
Actual Result	Got 401 Invalid username or password. The payload was completely blocked and no login happened. The server did not throw a database error or leak any internal information.
Status	PASS
Fix Needed	Nothing urgent right now, it is handling things fine. Just make sure parameterised queries keep being used on the backend going forward. If new authentication-related endpoints get added later they should follow the same pattern.

7. Detailed Findings → Threat, Risk and Dashboard Tests

This section covers the ten test cases that Rasanjana ran. The brief here was to cover three endpoint groups threat intelligence, risk assessment, and the dashboard. Six of those ten tests are marked as deferred which sounds bad on paper but the reason is simple: the threat intelligence and risk assessment backend services were not deployed in the test environment during our testing window. Rasanjana wrote up the full test plans for each of those anyway so they can be run as soon as those services come online.

7.1 RT-01 to RT-02 → Threat Intelligence Tests (Deferred)

RT-01 SQL Injection on /api/users/threats - DEFERRED. Plan was to inject classic SQL payloads through query parameters and check whether the database leaks data. Test could not be executed because the threats endpoint was not fully accessible during the testing window. Recommendation if it later fails: implement parameterised queries on every database interaction (Mitigation M-19 from the CY005 Map Mitigations document).

RT-02 RBAC End User access to /api/users/threats/:threatId - DEFERRED. Plan was to log in as an End User and try to view individual threat records that should only be accessible to Analyst-level users and above. Could not be completed because valid threat IDs and accessible detail records were not available during testing.

7.2 RT-03 — Unauthenticated Access to Threats Endpoint

STRIDE Reference	ST-03 - Soofing / Unauthenticated Access
Endpoint	GET /api/users/threats
Tool Used	Postman
Expected Result	401 Unauthorized for unauthenticated or invalid token requests
Actual Result	The threat list endpoint correctly rejected unauthenticated requests and returned HTTP 401 Unauthorized when no valid JWT token was provided. This confirms that authentication validation is properly enforced before backend processing occurs.
Status	PASS
Note	This is the same finding as AT-04. Rasanjana ran it independently and got the same result, which is good it confirms the fix is consistent.

7.3 RT-04 to RT-06 → Risk Assessment Tests (Deferred)

RT-04 Parameter tampering on risk assessment list — DEFERRED. Plan was to use Burp Suite to intercept the request as an End User and modify query parameters (role=admin, access=unrestricted, user_id=1) to see whether the server trusted client-supplied filter values. Could not be executed because the risk assessment endpoint did not return queryable records during testing.

RT-05 RBAC End User access to risk assessment detail — DEFERRED. Same blocker as RT-04. Plan was to use an End User token to fetch an Analyst-level assessment record.

RT-06 IDOR on risk assessment detail — DEFERRED. Plan was to modify the assessmentId in the URL (1, 2, 3, 10, 99, 100, 0, -1, abc) and see which IDs returned data that didn't belong to the authenticated user. Object-level authorisation is the thing being tested here. Could not run because predictable assessment records and accessible identifiers were not available.

7.4 RT-07 — Unauthenticated Access to Dashboard Overview

STRIDE Reference	ST-03 - Spoofing / Unauthenticated Access
Endpoint	GET /api/users/dashboard/overview
Tool Used	Postman
Expected Result	401 Unauthorized for both no-token and invalid-token attempts
Actual Result	The API returned HTTP 401 Unauthorized with the message No token provided, confirming that authentication protection is enforced correctly on the dashboard overview endpoint.
Status	PASS

7.5 RT-08 — Expired / Invalid Token on Dashboard

STRIDE Reference	ST-25 - Session Exploitation
Endpoint	GET /api/users/dashboard/overview
Tool Used	Postman
Expected Result	401 Unauthorized invalid or expired JWT tokens must be rejected
Actual Result	The server rejected the invalid/expired JWT token and returned HTTP 401 Unauthorized with the message Invalid token. JWT validation is working as it should.

Status	PASS
---------------	------

7.6 RT-09 — RBAC on Dashboard Charts

STRIDE Reference	ST-26 - Elevation of Privilege / RBAC Misconfiguration
Endpoint	GET /api/users/dashboard/charts
Tool Used	Postman
Expected Result	403 Forbidden or significantly limited data returned to End User full Analyst-level chart data must not be visible to an End User account
Actual Result	The API authenticated the analyst token successfully but returned HTTP 500 Internal Server Error with the message Error fetching dashboard charts. This confirms authentication protection is working correctly however the backend dashboard charts functionality failed internally during processing.
Status	PARTIAL PASS
Fix Needed	Authentication and RBAC protection are working. The dashboard charts endpoint itself needs proper exception handling so that internal errors don't leak through as 500 responses to authenticated users. Likely seeing this because the test environment has no chart data in the database.

7.7 RT-10 — Authenticated Analyst Access to Dashboard Overview

STRIDE Reference	ST-26 - Elevation of Privilege / RBAC Validation
Endpoint	GET /api/users/dashboard/overview
Tool Used	Postman
Expected Result	200 OK authenticated analyst users should successfully access authorised dashboard overview data through valid JWT authentication
Actual Result	The API authenticated the analyst JWT token successfully but returned HTTP 500 Internal Server Error with the message Error fetching dashboard overview. Authentication and RBAC protections are working but the underlying dashboard logic failed during processing most likely due to the empty test database.
Status	PARTIAL PASS
Fix Needed	Implement proper exception handling on the dashboard endpoints. When the database has no data the endpoint should return 200 with an empty array, not crash with a 500.

8. Detailed Findings → Hazard Endpoint Tests

This section covers the three hazard endpoint test cases Hayden ran. Quick note Hayden also did a full set of data ingestion endpoint tests earlier in the trimester (DI-01 to DI-04) which are part of his separate testing document on GitHub. The most important finding from that work was DI-03 where the ingestion endpoint accepted unauthenticated POST requests and returned 202 Accepted, meaning anyone on the network could push data into Phoenix without authenticating. That finding is included in the priority list in Section 10.

The hazard endpoint testing was complicated by the fact that the hazard table was empty in the test database during the testing window. So most requests crashed the endpoint with a 500 before authentication or input validation could be properly evaluated. The results below should be read with that backdrop.

For HZ-03 the test was actually re-run on my end because Hayden's analyst login was returning 500 after a backend merge. I ran the IDOR requests against /hazards/1, /2 and /999 using my own analyst token and gave the screenshots back to Hayden for his document.

8.1 HZ-01 → Unauthenticated Access to Hazards Endpoint

Endpoint	GET /api/users/hazards
Attack Tactic	Unauthenticated access
Tool Used	Postman
Expected Result	401 Unauthorized - unauthenticated clients must not be able to fetch application data
Actual Result	The server returned HTTP 500 Internal Server Error with the message Error fetching hazards. From the response alone we cannot say whether the authentication middleware correctly rejected the request before reaching the database, or whether the request got through and the database query crashed because the table was empty.
Status	PARTIAL PASS
Fix Needed	Enforce strict authentication on this endpoint. Unauthenticated requests should be rejected at the gateway with a 401 before any database call is made. Also worth fixing the underlying 500 an empty hazard table should return a 200 with an empty array.

8.2 HZ-02 → SQL Injection via Query Parameter

Endpoint	GET /api/users/hazards?id=' OR '1'='1
Attack Tactic	SQL injection via query parameter
Tool Used	Postman
Expected Result	400 Bad Request server should validate or sanitise the id parameter and reject the malformed input
Actual Result	The server returned HTTP 500 Internal Server Error with the message Error fetching hazards. Same situation as HZ-01 we cannot confirm from the response alone whether the input validation rejected the payload or whether the server just crashed.
Status	PARTIAL PASS
Fix Needed	Add explicit input validation on the id query parameter so malformed values are rejected with a 400 before any database call. Even if the underlying query is parameterised, the endpoint should reject obvious injection attempts at the entry point.

8.3 HZ-03 → Insecure Direct Object Reference

Endpoint	GET /api/users/hazards/{id}
Attack Tactic	IDOR modifying record ID to access other records
Tool Used	Postman
Test Steps	Logged in with analyst_test1 credentials, obtained a valid Bearer token, then sent GET requests to /api/users/hazards/1, /api/users/hazards/2 and /api/users/hazards/999 with the token
Expected Result	Server should properly validate object-level access controls so that authenticated users cannot view records they should not have access to
Actual Result	All three requests returned HTTP 500 Internal Server Error with the message Error fetching hazard. Because no actual records came back, it was not possible to confirm whether object-level authorisation is enforced there was nothing to compare across IDs.
Status	PARTIAL PASS
Fix Needed	Once hazard data is seeded into the test environment, this test should be re-run. The endpoint also needs to be reviewed in code to confirm that the authenticated user's identity is checked against the owner of the hazard record before any record is returned.

9. Detailed Findings → Metadata Endpoints and Infrastructure

Inderjeet's work was split in two halves. The first half was infrastructure stabilisation — getting the local Phoenix environment into a state where it could actually be tested against. The second half was the security audit of the six production metadata endpoints. Both halves matter to this report so I am including a summary of both.

9.1 Infrastructure Stabilisation (Phase 1 Work)

Before any meaningful testing could happen, the Phoenix backend had to actually run. Inderjeet documented two major blockers that he resolved during this phase.

Database schema conflict. The first time the backend was launched, the PostgreSQL container would start and then immediately terminate. Log inspection showed a conflict between `01_schema.sql` and `02_sample_data.sql` — specifically the sample data script was trying to insert into a `geo_location_id` column that the schema script had not created. The fix was to rename `02_sample_data.sql` to `.bak` so that the schema build could complete without the corrupted insert running.

Port 3001 conflict. The `.env` file specified Port 3001 for the API Gateway but Test-NetConnection on 3001 was returning False. A diagnostic scan showed that the service had defaulted to Port 3000. Initial testing had to be done against Port 3000 to match the active listener. After full Docker orchestration recovery, the gateway configuration was synchronised so that the final production endpoints exposed on Port 3001 (which is the port everyone in the team ended up using).

These two fixes are why the rest of us could actually run our tests. Without the stable Port 3001 baseline the AT, RT and HZ test cases above would not have been possible.

9.2 Honest Note → Mock Routes vs Production Routes

Inderjeet's first round of testing was carried out against routes that he had manually added to `user.routes.ts` (`router.post('/location')` and `router.post('/metadata')`) to simulate the production environment locally while the Port 3001 conflict was still being resolved. I gave him feedback that the testing needed to be done against the real production endpoints, not the mock routes. He took that feedback on, migrated the testing across to the six real `/api/users/meta` routes on Port 3001, and re-ran everything. The results below are from the second round against the real endpoints.

9.3 MD-01 to MD-06 → SQL Injection Sweep Across Six Metadata Endpoints

Inderjeet wrote a PowerShell script that fired the same SQL injection payload at all six production metadata endpoints in sequence. The payload was a classic destructive injection combined with an overflow constraint:

Payload: Melbourne'; DROP TABLE hazard_event; -- followed by 110 'X' characters

The six endpoints under test:

- GET /api/users/meta/locations
- GET /api/users/meta/event-statuses
- GET /api/users/meta/linked-event-types
- GET /api/users/meta/seasons
- GET /api/users/meta/reference-days
- GET /api/users/meta/reference-times

Result across all six endpoints: SUCCESS - Input was BLOCKED by the Gateway. The API Gateway intercepted the malicious queries on every endpoint, dropped the payload, and returned a safe termination state. None of the requests reached the PostgreSQL data layer. The gateway is doing exactly what it should be doing defence in depth at the perimeter.

Status: **PASS** across all six endpoints (MD-01 to MD-06).

Recommendation: Working correctly. The gateway's blocking ruleset should be checked into version control and reviewed periodically gateway-level filters tend to drift over time as new rules get added without anyone tracking the originals. Inderjeet also recommended adding express-validator middleware to enforce length (Rule 9) and enumerated value (Rule 2) constraints at the route handler level for extra defence.

9.4 MD-07 → Unauthenticated Access Layer Audit

The second part of the metadata audit was checking whether the same six endpoints would accept clean GET requests with no Authorization header at all. The PowerShell script sent structurally valid GET requests without any auth header to each of the six routes.

Result: requests were handled and stopped by the gateway, but the status code variable in the PowerShell exception block was left unpopulated. So we can see that the gateway is dropping the requests, but the

exact numeric status code (401 versus something else) was not captured cleanly because of a quirk in how Express formats the exception object. The gateway is doing its job structurally, but the exception tracking on the client side needs a small fix to format the downstream status string cleanly.

Status: **PARTIAL PASS.** The boundary is enforced but the test evidence is not as clean as we would like. A second run with curl or Postman would confirm the exact status code returned.

9.5 Inderjeet's Action Items for Week 10

These were Inderjeet's recommendations from his own report, listed here because they are still valid:

- Middleware implementation → integrate express-validator for Rule 9 (length) and Rule 2 (enumerated values)
- Sanitisation layer → add a global filter to remove SQL special characters like ; -- and) from the JSON body
- Environment sync → fix the .env update so that PORT is respected and the gateway never defaults to 3000
- Database seeding → correct 02_sample_data.sql so it matches the current schema and can be restored

10. Vulnerabilities Found → Priority List

This is the consolidated list of every active vulnerability found across the team, ranked by severity. The severity is based on how serious the impact would be if the issue was exploited combined with how easy the exploit actually is in practice.

ID	Finding	Severity	Found By	Owner
DI-03	Unauthenticated POST to /api/ingestion/stream returns 202 Accepted - anyone can push data into the system	CRITICAL	Hayden	Ingestion service
AT-01	No rate limiting on login endpoint — 15 wrong-password attempts allowed with no lockout, no 429, no delay	HIGH	Himanshu	Gateway / User service
AT-03	Registration endpoint accepts role field from client — admin can create unlimited new admin accounts by adding role admin to the body	HIGH	Himanshu	User service
HZ-01	Hazard list endpoint returns 500 for unauthenticated requests instead of 401 — auth middleware may not be applied at gateway level	MEDIUM	Hayden	Hazard service
HZ-02	Hazard endpoint returns 500 on malformed query parameters - input validation missing before database call	MEDIUM	Hayden	Hazard service
RT-09/10	Dashboard endpoints crash with 500 when database is empty - should return 200 with empty array instead	LOW	Rasanjana	Dashboard service
MD-07	Exception tracking on unauthenticated metadata audit did not capture the exact status code cleanly - minor evidence gap	LOW	Inderjeet	Test framework

10.1 The Three Items That Need Immediate Attention

Of all the findings above three of them stand out as urgent they should be fixed before the platform moves any further down the development pipeline.

1. Unauthenticated ingestion (DI-03) - CRITICAL. This is the worst finding in the whole report. Anything that can reach the ingestion endpoint can push data into the Phoenix system without identifying itself. Until this is fixed, the threat data inside Phoenix cannot be fully trusted, because we cannot tell which

records came from authenticated services and which came from outside the trust boundary. If the ingestion endpoint is ever exposed to a wider network this becomes an immediate data integrity problem.

2. No rate limiting on login (AT-01) - HIGH. Phoenix usernames follow predictable patterns (admin1, analyst_test1) which makes this even worse. Combined with no rate limiting, an attacker only needs a wordlist and a script to start guessing. This should be fixed before any non-test users are added to the platform.

3. Role field accepted from client (AT-03) - HIGH. The fix here is small. Default new registrations to the lowest privilege role and create a separate admin-only endpoint for role promotion. The impact of not fixing it is much bigger than the cost of fixing it — any compromised admin account can quietly create shadow admin accounts that are hard to detect later.

11. Recommendations

Below are the recommendations from the assessment, grouped by how urgently they need to be done. Each item points back to the test case or tests it came from.

11.1 Immediate → Should Be Fixed Before Next Release

- Implement rate limiting on POST /api/users/auth/login. Reasonable starting point is 5 failed attempts in a 15 minute window from the same IP or against the same username, followed by a 429 response. (Source: AT-01)
- Add authentication middleware to POST /api/ingestion/stream. Use a service-level token rather than a generic user token, scoped to the ingestion role only. (Source: DI-03)
- Remove the role field from the body of POST /api/users/auth/register. Default all new registrations to the lowest privilege role and create a separate admin-only endpoint for role promotion. (Source: AT-03)

11.2 Short Term → Within The Next Sprint

- Replace 500 Internal Server Error responses on the hazard and dashboard endpoints with proper status codes. Empty results should return 200 with an empty array. Malformed input should return 400. (Source: HZ-01, HZ-02, RT-09, RT-10)
- Add input validation on the id query parameter for /api/users/hazards so malformed values get rejected at the entry point before the database is touched. (Source: HZ-02)
- Audit all /api/users/* routes to confirm authentication middleware is applied consistently from a single central place. The fact that the threats endpoint was missed initially (AT-04) suggests this is not currently the case.
- Bring the gateway-level injection filter ruleset into version control and review it periodically. Right now it is doing a great job blocking SQL injection on the metadata endpoints (MD-01 to MD-06) but if no one tracks the rules they tend to drift over time. (Source: Inderjeet's recommendations)

11.3 Medium Term → Before The Next Full Audit

- Re-run the six deferred RT tests (RT-01, RT-02, RT-04, RT-05, RT-06) once the threat intelligence and risk assessment services are available in the test environment. These represent a real coverage gap right now.

- Re-run HZ-03 IDOR once hazard data is seeded into the test database. We could not validate object-level authorisation without comparison records.
- Re-run DI-01 to confirm whether analyst tokens are correctly blocked from pushing data through the ingestion endpoint.
- Add express-validator middleware to enforce length (Rule 9) and enumerated value (Rule 2) constraints at the route handler level, on top of the gateway filtering that is already there. (Source: Inderjeet)
- Fix the database seeding - correct 02_sample_data.sql so it aligns with the current schema. This will let the test environment have realistic data, which in turn means several partial passes can be retested and either upgraded to pass or downgraded to fail with clean evidence. (Source: Inderjeet)

12. Backend Team Coordination

One thing that is worth calling out in this report is how much the working relationship with the backend team mattered to the quality of the testing. Trung in particular was responsive throughout the trimester and was willing to investigate every finding we raised, push back where the evidence was thin, and provide his own evidence when our test setup was incorrect.

Two of my own test cases moved from FAIL to PASS during the testing window because of this back-and-forth:

- AT-02 (logout) initially failed because we were calling `/api/users/auth/logout` without the `userId` path parameter and getting a 404. Trung clarified the correct URL format `/api/users/auth/logout/{userId}` with screenshots. After retesting the endpoint passed cleanly.
- AT-04 (unauthenticated threats access) initially failed because the endpoint was returning 200 OK with data when called with no token. The backend team patched the authentication middleware on that route and a retest confirmed the fix - it now correctly returns 401.

There was also a moment mid-trimester when Trung merged the dev-backend branch into dev which broke local logins for a while because the database test accounts were affected. We worked through it together — I rebuilt my local containers, the test accounts came back online, and everyone was able to continue testing. This kind of coordination is the difference between a security report that finds real issues and one that mostly finds environment problems.

This is the model that should be carried forward into future testing rounds. Findings should always be shared with the development team in time for them to investigate before the report is finalised, and the report should reflect the most current state of the system at handover.

13. Conclusion

The Phoenix backend is in a reasonable state from a security point of view, but it is not finished. The defensive controls that exist are working well — SQL injection is blocked across login and all six metadata endpoints, JWT validation is enforced on dashboard endpoints, logout properly invalidates refresh tokens, and the threats endpoint that was previously open has been patched. These are real wins.

But the gaps that remain are not small. The login endpoint has no rate limiting. The data ingestion endpoint will accept data from anyone who can reach it. The registration endpoint trusts the client to choose its own role. These are exactly the kinds of issues that get exploited in real-world breaches, and they should be treated as blockers for any wider release of the platform.

It is also worth being upfront that a meaningful slice of our planned testing could not be executed because the underlying services were not deployed in the test environment. The threat intelligence and risk assessment endpoints in particular were not testable during this trimester. The next CY015 team picking up this work should start there.

Overall I am happy with what the team delivered. Twenty-eight test cases, three real vulnerabilities identified, multiple backend fixes confirmed in retesting, full documentation with evidence for everything that was run, and a working relationship with the backend team that made the whole thing possible. That is a good place to hand the project off from.

14. Acknowledgements

Thanks to Trung from the backend team for being available throughout, for taking every finding seriously, and for pushing back where it was warranted. That responsiveness is the single biggest reason this report is as accurate as it is.

Thanks to Jessica and Divye for the project guidance and ongoing support throughout the trimester.

Thanks to Rasanjana, Hayden and Inderjeet - the testing work in sections 7, 8 and 9 of this report is theirs. I just pulled it together.

— Himanshu Khanna

Sub-Team Leader, CY015

Project Phoenix — Cyber Security Sub-Team

May 2026